

Codifica e codici

venerdì 27 febbraio 2026 11:23

Come già detto i moderni computer elaborano vari tipi di file audio, immagini, ma alla base di questi file vi sono le sole informazioni riconosciute: due livelli di informazione alto/basso oppure 0/1 oppure acceso/spento, quindi per manipolare qualunque tipo di dato bisogna scegliere una rappresentazione adeguata: **la codifica** (alfabeto finito associato a qualche configurazione). Nella codifica bisogna distinguere **numeri e numerali**: i primi sono delle entità astratte che esistono a prescindere dalla rappresentazione, mentre i secondi rappresentano una sequenza di caratteri in un qualsiasi sistema posizionale. In virtù di ciò si possono definire **sistemi numerici posizionali**: sequenze di numerali:

$$(x)_b = (a_n a_{n-1} \dots a_1 a_0, c_1 c_2 c_3 \dots) = \sum_{i=0}^n (a_i \cdot b^i) + \sum_{k=1}^{\infty} c_k b^{-k}$$

Di questi sistemi numerici è possibile fare la conversione da una base ad altra sia per la parte intera (sx della virgola) sia per la parte frazionaria (dx della virgola). Nel primo caso si divide il numero per la base corrispondente, mentre per la parte frazionaria si moltiplica per la base. Si parte dalla fine a ritroso. Si usano principalmente tre basi: base 2, base 8, base 16: base 2 $\rightarrow \{0,1\}$, base 8 $\rightarrow \{0,1,2,3,4,5,6,7\}$, base 16 $\rightarrow \{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F\}$. Possibile passare da base ad altra, raggruppando opportunamente le cifre: 3 se base 8, 4 se base 16. Vediamo un esempio:

8 $\rightarrow$ 2	2 $\rightarrow$ 8
65 $\rightarrow$ 101 110	101 100- >54
17 $\rightarrow$ 001 111	110 010- >62

16 $\rightarrow$ 2	2 $\rightarrow$ 16
AF $\rightarrow$ 1010 1111	1011 1001 $\rightarrow$ B9
1C $\rightarrow$ 0001 1100	0000 1001 $\rightarrow$ 09

Quindi in generale si vede il valore di k intero tale che:

Intervallo di rappresentazione $[0, 2^k - 1]$
Numero di bit necessari $k = \log(\text{base})n$

Questo discorso vale solo per i numeri positivi, ma visto che i processori attuali possono manipolare anche i numeri negativi, bisogna implementare le stesse logiche: **Rappresentazione in modulo e segno, complemento ad uno e complemento a due**.

Nella prima si va a vedere la cifra più significativa (a sx) e se è "0" è positivo, mentre se "1" è negativo:

10001100	-12
00001100	+12
00000000	+0
10000000	-0

Occhio alla rappresentazione doppia dello 0!!!

Quindi questa rappresentazione è inefficiente se circuiti più complessi, aumentando i costi e diminuendo le prestazioni.

Nel secondo si ha sempre la doppia rappresentazione dello "0", bit più significativo ancora quello più a sx, ma questo numero si ottiene invertendo tutti i bit:

00001100	+12
11110011	-12

Quindi intervallo di rappresentazione: $[-(2^{(n-1)}-1), 2^{(n-1)}-1]$

Da notare che in questa rappresentazione potrebbe verificarsi la non veridicità del risultato in quanto si ha effetto **end-around-carry**:

$$\begin{array}{r} 1111 \ 1110 \ + \\ 0000 \ 0010 \ = \\ \hline 10000 \ 0000 \end{array} \quad \begin{array}{r} -1 \ + \\ 2 \ = \\ \hline 0 \end{array}$$

Quindi devo fare somma ulteriore: riporto al risultato

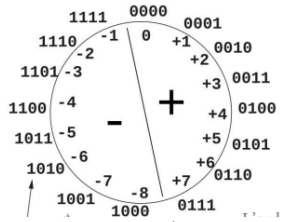
Nell'ultima invece si ha la rappresentazione più flessibile semplice ed efficiente che un calcolatore possa computare. Si ottiene facendo così: partendo dal bit meno significativo, si lasciano inalterati tutti i bit fino al primo "1" **compreso** e poi si complementa bit a bit, oppure si complementa bit a bit e si aggiunge "1":

Primo modo	1011 1001	0100 0111		0100 0111
Secondo modo	1011 1001	0100 0110	+1	0100 0111

Per quanto riguarda le somme / sottrazioni in questo ambito può essere svolta ignorando il segno, quindi possibile usare stesso circuito per due operazioni.

Quindi intervallo di rappresentazione : $[-(2^{n-1}), 2^{n-1}-1]$

Riepilogando il complemento a 2 :



Da notare che con questa rappresentazione posso rappresentare sia i numeri positivi che i numeri negativi, ma da notare che il numero 8 (1000) non ha la controparte positiva : i positivi da 0 a 7 , mentre i negativi da 8 a 15 . Infatti il valore massimo è 2^{n-1} , il quale "-1" è dovuto alla rappresentazione dello "0" che leva il posto al "+8". Ricordiamo le operazioni di somma e sottrazione le quali vengono effettuate normalmente, ignorando il riporto più significativo. Ma attenzione al risultato : può capitare che il risultato superi il numero di cifre consentite, portando così il sistema in una condizione di

overflow:

1. Se sommo due numeri opposti il risultato è corretto
2. Se sommo due numeri positivi il risultato eccede la rappresentazione
3. Se sommo due numeri negativi il risultato eccede la rappresentazione

Per accorgermi se sono in questa condizione, devo vedere gli ultimi 2 riporti : se sono concordi va bene, altrimenti ho avuto un'inversione di segno durante l'operazione.

Similmente a questa divisione vi è un'altra rappresentazione : **Rappresentazione in eccesso** , ovvero in questa rappresentazione si sceglie opportuno valore di "0" : ovvero il valore k che è uguale a 2^{n-1} , dividendo così l'intervallo di numerazione in 2 sotto intervalli. In questa rappresentazione , chiamata anche offset binary , è possibile traslare lo "0" in avanti di k valori, avendo così la rappresentazione dello "0" unica e con cifra più significativa="1". Da notare che si mantiene ordinamento numero/numerales .

$$x' = x + k \quad x = x' - k$$

Con intervallo di rappresentazione pari a : $[-2^{n-1}, 2^{n-1}-1]$.

Ampliamo ora il nostro intervallo di rappresentazione : usiamo la **rappresentazione in virgola mobile** : ovvero possiamo rappresentare un'intervallo maggiore a parità di numero di bit. La gestione di questa rappresentazione, viene gestita da un'apposita circuiteria molto complessa e laboriosa : **FPU (floating ponit unit)** . Il dettaglio : **questa rappresentazione permette di spostare la virgola .**

$$12,345 = \underbrace{1,2345}_{\text{mantissa}} \times \underbrace{10^1}_{\text{base}} \quad \text{esponente}$$

Questa rappresentazione segue lo standard IEEE754 a 32 bit:



Usando esponente come eccesso $e = E + 127$.

Da notare che il segno ha un solo bit (0-> positivo, 1->negativo), l'esponente ne ha 8 mentre la mantissa i 23 restanti. La mantissa è rappresentata come $(1.x)$. Quindi un numero in questo formato viene scritto così:

$$(-1)^s \cdot 2^{e-127} \cdot 1.m$$

Vediamo un esempio : $(-53,1)$ base 10 :

1. Convertire il numero in binario
 - a. $53 \rightarrow 110101$
 - b. $0.1 \rightarrow 00011$
 - c. $-110101,00011001100110011$
2. Sposto la virgola
 - a. $-1,1010100011001100110011 \cdot (2^5)$
3. Estrapolo i valori:
 - a. $s=1$
 - b. Esponente: $5+127=132$
 - c. Mantissa= 1010100011001100110011
4. Rappresentazione dell'esponente
 - a. $132 = 10000100$
 - b. Se minore di 8 aggiungo "0" a sx
5. Codifica finale
 - a.

1	1000100	1010 1001 0011 0011 0011 0011
---	---------	-------------------------------

Quindi è possibile cambiare la rappresentazione della parola, quindi di una stessa parola si può cambiare il contesto:

Quindi la parola precedente diventa in hex : C2546666

Questi tipi di numeri di dicono normalizzati. In antitesi a questi ci sono i denormalizzati, i quali seguono la stessa logica, ma sono quelli che rappresentano numeri vicini allo 0, omettendo così la parte "0" del numero .

Da notare che in questa notazione la rappresentazione dello "0" è doppia, così come "infinito" (attenzione ad overflow) e così come i NaN (not a number) . In dettaglio :

e	m	Tipo di valore
$[1,254]$	qualsiasi	$(-1)^s \cdot 2^{e-127} \cdot 1.m$ (numeri normalizzati)
0	$\neq 0$	$(-1)^s \cdot 2^{-126} \cdot 0.m$ (numeri denormalizzati)
0	0	$(-1)^s \cdot 0$ (zero con segno)
255	0	$(-1)^s \cdot \infty$ (infinito con segno)
255	$\neq 0$	NaN

Per quanto riguarda la gestione dei NAN si hanno varie casistiche, sollevando delle eccezioni, ovvero delle condizioni di errore non previste che devono essere trattate opportunamente :

1. Operazione invalida
 - a. Operazione non corretta
2. Overflow
 - a. Numero eccede la rappresentazione
3. Divisione per zero:
 - a. Diviso per zero
4. Underflow:
 - a. Overflow per risultati troppo piccoli
5. inesatto:
 - a. Errore di arrotondamento

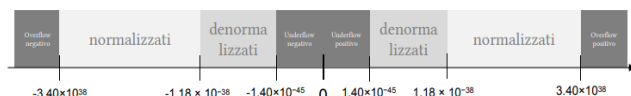
Vediamo ora i valori tipici della mantissa valori (normalizzati):

1. Esponente massimo : -126
 - a. Mantissa minima : 2^{-126}
2. Esponente minimo : 127
 - a. Mantissa massima : $2^{127} \cdot (2 - 2^{-23})$

Vediamo ora i valori tipici della mantissa valori (denormalizzati):

1. Esponente : 126
 - a. Mantissa minima : $2^{-126} \cdot 2^{-23} = 0 \cdot 2^{-149}$
 - b. Mantissa massima : $2^{-126} \cdot (1 - 2^{-23})$

Vediamo ora come sono disposti i numeri negli intervalli , ma usando la virgola mobile , quindi approssimando un numero, può capitare che vi sia un **errore di approssimazione** . Questi errori sono dovuti sia alla modalità con cui sono sparsi ,sia al sistema di numerazione usato.



Da notare che la maggior parte dei numeri , o meglio della densità, sono compresi tra -1 e 1 e tra 65563 e 131072. Come vedere l'errore commesso nell'approssimazione ??? Secondo due modi : **errore assoluto ed errore relativo**. Il primo dice quanto mi sono discostato da informazione originale, mentre il secondo dice se errore commesso è piccolo o grande .

$$\varepsilon_A = x - x'$$

Mentre per vedere il numero delle cifre affette da errore si usa il logaritmo in base :

$$-\log_{10} \varepsilon_R$$

Vediamo un esempio :

Consideriamo $x = 3.5648722189$ e supponiamo di volerlo rappresentare utilizzando soltanto quattro cifre decimali:

$$x \approx \bar{x} = 3.5648$$

Con questa approssimazione, otteniamo un errore relativo pari a:

$$\varepsilon_R = \frac{x - \bar{x}}{x} = \frac{3.5648722189 - 3.5648}{3.5648722189} = 0.000020258$$

Il numero di cifre affidabili di $\bar{x}$ è pari a:

$$-\log_{10} (0.000020258) = 4.69$$

Riepilogando : con n bit possiamo creare una qualsiasi codifica di una qualunque parola, come per esempio con 3 bit possiamo codificare i giorni della settimana, assegnando ad ogni giorno una codifica :

Lunedì	000
Martedì	001
Mercoledì	010
Giovedì	011
Venerdì	100
Sabato	101
Domenica	110

Ma attenzione!!! Finora abbiamo supposto che esista una sola codifica per ciascun elemento che vogliamo rappresentare, ma può capitare che vi sia un codifica particolare : **quella ridondante** : quella codifica che usa più bit di quelli concessi per la rappresentazione. Vediamo un esempio : dati N elementi da rappresentare e n cifre possibile e $m = \log_2 N$, si ha che:

1. $n = m$
 - a. Codice irridondante (va bene).
 - b. Distanza di humming =1 (va bene)
2. $n > m$
 - a. Codice ridondante
 - b. Le $k = n - m$ cifre aggiuntive sono chiamate **cifre di controllo**.
 - c. **Distanza di humming** $\leq h - 1$

- d. In questi codici si riesce a trovare e/o correggere errori grazie alla ridondanza: un esempio può essere l'inversione di bit, la quale può essere dovuta ad errori di trasmissione e/o particelle che colpiscono le celle di memoria:



Vediamo ora un primo modo di vedere se la codifica è "giusta": Usiamo il codice di hamming: numero di bit diversi tra due parole:

$$D(10010, 01001) = 4$$

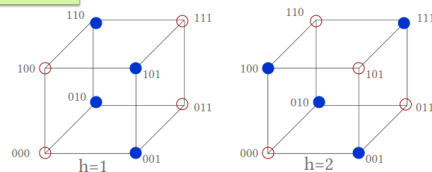
$$D(11010, 11001) = 2$$

In generale la distanza di hamming è uguale al valore minimo della distanza tra due parole :

$$H = \min(d(x, y)) \text{ , per qualunque parola } x \neq \text{parola } y$$

Andiamo ora a vedere i codici rivelatori di errore : in generale questi codici possono rivelare uno o più bit di errore, ed in base al quantitativo di bit si hanno diverse rappresentazioni , ognuna delle quali ha parole ammesse ed altre no :

Elemento	Codice 1	Codice 2
A	000	000
B	100	011
C	011	101
D	111	110



○ Parole del codice (legali)

● Parole non appartenenti al codice

Ricordiamo che i moderni calcolatori interagiscono con i dati attraverso il codice ascii , ovvero una rappresentazione ad 8 bit di ogni singolo carattere.

Addentriamoci ora veramente nei codici rivelatori di errori : BCD (Binary decimal code), Codice Gray , Codice di parità. Il primo è un codice ridondante , che rappresenta le 10 cifre decimali (0-9) usando 4 bit . Ognuna di queste cifre è codificata indipendentemente : di queste 4 cifre 2 sono memorizzate in un singolo byte . Da notare che con questa codifica circa 1/6 dei bit della codifica sono sprecati:

Base 10	BCD	Base 10	BCD
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

Passiamo ora al secondo : questo codice inventato da Grey , ha una lunghezza fissa , ma è caratterizzato da una codifica del bit tale che tra due numeri adiacenti cambi un solo bit :

Base 10	Gray	Base 10	Gray
0	000	4	110
1	001	5	111
2	011	6	101
3	010	7	100

Ma ha una pecca : non distingue le configurazioni transitorie da quelle corrette.

Vediamo ora l'ultimo: questo è un semplice codice ridondante con ridondanza $h=2$, e si ottiene aggiungendo una cifra di parità al codice irridondante:

Codice irridondante	Parità	Disparità
000	000 0	000 1
001	001 1	001 0
010	010 1	010 0
011	011 0	011 1
100	100 1	100 0
101	101 0	101 1
110	110 0	110 1
111	111 1	111 0

Due tipologie : parità vale 1 se il numero di "1" nella codifica irridondante è dispari e disparità se il numero di "1" nella codifica è pari: vediamo un esempio:

Ricevuto	Parità	Segnale di errore
1010	uguale a zero	OK
1110	diversa da 0	ERRORE
1111	uguale a zero	OK

Torniamo ad un esercizio sulla virgola mobile : Convertire 0.3 in virgola mobile:

Si supponga di avere mantissa a 5 bit e esponente a 4 bit

Numero	Moltiplicato per 2	Resto
0.3	0.6	0
0.6	1.2	1
0.2	0.4	0
0.4	0.8	0
0.8	1.6	1

Qui il processo si interrompe che si ripete , quindi il risultato è :

0.01001(si ripetono)

Riportando il risultato nella forma canonica $1.x$ si ha che :

$1,001(\text{periodico}) \cdot 2^{(-2)}$

Il quale riportando la mantissa a 5 bit diventa :

$1,0011001 \cdot 2^{(-2)}$

Riassumendo quindi :

$N=7$

Esponente = $-2+7=+5 \rightarrow 0101$

$S=0$

Andiamo a vedere ora l'errore assoluto e quello relativo :

$$\varepsilon_A = x - x' \quad \varepsilon_R = \frac{x - x'}{x}$$

Il primo vale :

$$0.3 - (2^{(-2)}) (1 \cdot 2^{-3} + 1 \cdot 2^{-4}) = 0.003125$$

(qui si prende il valore normalizzato , o meglio la parte della mantissa)

Il secondo invece :

$$(0.3 - 0.296875) / 0.3 = 0.104$$

Mentre per quanto riguarda il numero corretto di cifre si fa il log:

$$-\log(0.104) = 2 !!!$$

Vediamo ora un metodo che serve per costruire codici a distanza $h \geq 3$: **codice di humming**: data una qualunque sequenza di bit da trasmettere, vi si associano n bit di parità , in modo che il messaggio venga codificato.

Posizione cifra	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Dato codificato	p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15
Copertura bit di parità	p1	✓		✓			✓		✓			✓		✓		✓			✓	
	p2		✓	✓			✓	✓		✓	✓		✓	✓				✓	✓	
	p4				✓	✓	✓	✓				✓	✓	✓	✓					✓
	p8								✓	✓	✓	✓	✓	✓	✓					
	p16															✓	✓	✓	✓	✓

Vediamo ora in dettaglio come funziona:

- Ipotesi fondamentale :
 - $N+1 \leq 2^n$
- Bit di parità o controllo sono nelle posizioni delle potenze di 2 : 1,2,4,8 ecc ecc

Esempio :

- Codificare 10011010
- $N=8$ per trovare k si va a tentativi:
 - $K=3$ non rispetta disequazione : $9 \leq 7$ FALSO
 - $K=4$ rispetta disequazione : $9 \leq 16-4$ Vero $\rightarrow$ prendiamo questo valore di k
 - $M=n+k \rightarrow 8+4=12$
- Si scrive il messaggio nel seguente modo includendo i bit di parità :
 - $**1*0001*1010$ [^], dove gli "*" sono i bit di parità nelle posizioni delle potenze di 2, assumendo la cifra meno significativa a sinistra**
- Vediamo la rappresentazione in binario delle configurazioni:

1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010

11	1011
12	1100

Dopo mi fermo perché uso 12 bit massimo!

5. Dato che abbiamo 4 bit di parità dobbiamo trovare la parità di tutte le k parole : per ogni bit dobbiamo vedere il bit n-esimo settato ad "1" se presente :
- P1 = 1,3,5,7,9,11
 - Di questa configurazione prendiamo i valori nella [^] corrispondenti , ignorando il primo numero in cui compare :
 - Nel nostro caso : 1+0+1+1+1
 - P2 = 2,3,6,7,10,11
 - Nel nostro caso : 1+0+1+0+1
 - P4 = 4,5,6,7,12
 - Nel nostro caso : 0+0+1+0
 - P8 = 8,9,10,11,12 non vado oltre perché ho solo 12 bit
 - Nel nostro caso : 1+0+1+0
 - Di ognuna di queste configurazioni facciamo or tra gli addendi : se totale è pari il bit di parità vale 0 , altrimenti 1
 - In questo esempio ho come sequenza 0110
 - Assemble il messaggio finale inserendo al posto di "*" i bit corrispondenti :**
 - 011100101010

Vediamo la correzione dell'esercizio per casa : dati due numeri a 16 bit fare il complemento a 2, fare la somma ed trasformare la somma in ottale :

A=2176	B=-2040
--------	---------

Cominciamo con il trasformare i numeri in binario in valore (il secondo):

2176	0
1088	0
544	0
272	0
136	0
66	1
33	0
16	1
8	0
4	0
2	0
0	1

Dato che la rappresentazione della parola è a 12 bit, ne dobbiamo aggiungere 4 :

0000	1000	1010	0000
------	------	------	------

Analogamente per b, ma considerando il modulo :

2040	0
1020	0
510	0
255	0
127	0
63	1
31	1
15	1
7	1
3	1
1	1
0	1

Dato che abbiamo calcolato il valore in modulo, dobbiamo trasformarlo nel complemento a 2 : prima complemento tutti i bit a partire dal primo "1" e poi aggiungiamo "1" :

0000	0111	1111	1000
------	------	------	------

Complemento a 2 :

1111	1000	0000	0111
			+1
1111	1000	0000	1000

Quindi è la rappresentazione in complemento a 2 del numero -2040 !

Facciamo ora la somma : **se signed non c'è overflow** , in quanto i numeri sono opposti , altrimenti :

0000	0111	1111	1000	
				+
1111	1000	0000	1000	=
10000	0000	0000	0000	

Infine vediamo la parola in ottale (raggruppo i bit in gruppi da 3):

1000000000000 -> 001 000 000 000 000 -> 1 0 0 0 0

Esempio :

- Codificare 0110000
- N=7 per trovare k si va a tentativi:
 - K=3 non rispetta disequazione : $8 \leq 7$ FALSO
 - K=4 rispetta disequazione : $8 \leq 16-4$ Vero -> prendiamo questo valore di k
 - $M=n+k \rightarrow 7+4=11$
- Si scrive il messaggio nel seguente modo includendo i bit di parità :
 - **0*110*000 [^], dove gli "*" sono i bit di parità nelle posizioni delle potenze di 2, assumendo la cifra meno significativa a sinistra**
- Vediamo la rappresentazione in binario delle configurazioni:

1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011

Dopo mi fermo perché uso 11 bit massimo!

- Dato che abbiamo 4 bit di parità dobbiamo trovare la parità di tutte le k parole : per ogni bit dobbiamo vedere il bit n-esimo settato ad "1" se presente :
 - P1 = 1,3,5,7,9,11
 - Di questa configurazione prendiamo i valori nella [^] corrispondenti , ignorando il primo numero in cui compare :**
 - Nel nostro caso : 0+1+0+0+0
 - P2 = 2,3,6,7,10,11
 - Nel nostro caso : 0+1+0+0+0
 - P4 = 4,5,6,7
 - Nel nostro caso : 1+1+0
 - P8 = 8,9,10,11 non vado oltre perché ho solo 11 bit
 - Nel nostro caso : 0+0+0
 - Di ognuna di queste configurazioni facciamo or tra gli addendi : se totale è pari il bit di parità vale 0 , altrimenti 1**
 - In questo esempio ho come sequenza 0110
 - Assemblo il messaggio finale inserendo al posto di "*" i bit corrispondenti :**
 - 11001100000
- Poniamo caso che il messaggio ricevuto sia : 11001100100 (inclusi i bit di parità) , come facciamo a vedere la posizione dove c'è stato il bit flip ? Ripetiamo il procedimento ,ma con la nuova parola :
Esempio :
 - decodificare 11001100100
 - N=8 per trovare k si va a tentativi:
 - K=3 non rispetta disequazione : $9 \leq 7$ FALSO
 - K=4 rispetta disequazione : $9 \leq 16-4$ Vero -> prendiamo questo valore di k
 - $M=n+k \rightarrow 7+4=11$
 - Vediamo la rappresentazione in binario delle configurazioni:

1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011

12	1100
----	------

Dopo mi fermo perché uso 12 bit massimo!

4. Dato che abbiamo 4 bit di parità dobbiamo trovare la parità di tutte le k parole : per ogni bit dobbiamo vedere il bit n-esimo settato ad "1" se presente :
 - a. $P1 = 1,3,5,7,9,11$
 - i. Di questa configurazione prendiamo i valori nella [^] corrispondenti , ignorando il primo numero in cui compare :
 - ii. Nel nostro caso : $0+0+0+1$
 - b. $P2 = 2,3,6,7,10,11$
 - i. Nel nostro caso : $0+1+0+0+0$
 - c. $P4 = 4,5,6,7$
 - i. Nel nostro caso : $1+1+0$
 - d. $P8 = 8,9,10,11$ non vado oltre perché ho solo 11 bit
 - i. Nel nostro caso : $1+0+0$
 - e. Di ognuna di queste configurazioni facciamo or tra gli addendi : se totale è pari il bit di parità vale 0 , altrimenti 1
 - f. **Aggiungo ulteriore parità :**
 - i. In questo esempio ho come sequenza 1001
 - g. **Assemblo il messaggio finale inserendo al posto di "*" i bit corrispondenti :**
 - i. 11001100000
 - h. Lo confronto con il messaggio di partenza :
 - i. 11001100100
 - i. Converto in decimale il valore della seconda parità :
 - i. $(1001)_{base\ 2} \rightarrow (9)_{base\ 10}$, quindi il bit flippato è in questa posizione.